

DocumentoStudio1

Guida allo studio di `asa-deliveroo-agent`
Architettura, stato attuale e direzioni di evoluzione

Progetto d'esame ASA — UniTn

Repository: <https://github.com/Giambac/asa-deliveroo-agent>

11 giugno 2026

Indice

1	Scopo di questo documento	2
2	Il progetto in sintesi	2
2.1	Requisiti d'esame coperti	2
2.2	Stato attuale (validato)	2
3	Architettura generale	2
3.1	Il ciclo di controllo BDI	2
3.2	Principi di design (da rispettare nelle evoluzioni)	3
4	Mappa del repository	3
5	Il nucleo BDI in dettaglio	5
5.1	BeliefBase — lo stato del mondo	5
5.2	OptionGenerator — che cosa è possibile	5
5.3	IntentionRevision — impegno con isteresi	5
5.4	Intention e PlanLibrary — dal volere al fare	5
5.5	ActionExecutor — serializzazione e semaforo	6
5.6	AgentLoop — il collante	6
6	Navigazione deterministica	6
6.1	GridGraph — la mappa come grafo orientato	6
6.2	PathPlanner — BFS e helper di scoring	7
7	PDDL	7
8	Le strategie	7
8.1	L'interfaccia	7
8.2	Le quattro strategie esistenti	7
8.3	Come aggiungere una nuova strategia (passo-passo)	8
9	L'Agente B: livello LLM	8
9.1	Flusso	8
9.2	MissionInterpreter — doppio binario	9
9.3	Tools (per esperimenti futuri)	9
10	Coordinamento A–B	9

11 Metriche, log ed esperimenti	10
12 Storia della validazione (e lezioni)	10
12.1 Il bug dei “parcel fantasma”	10
12.2 Fix del decay (contributo Codex)	10
12.3 Lezione generale	10
13 Step futuri	10
14 Scelte aperte e parametri da esplorare	11
14.1 Manopole esistenti (nessuna modifica al codice richiesta)	11
14.2 Idee strategiche da prototipare	11
14.3 Domande aperte da verificare a runtime	12
15 Regole di lavoro e comandi rapidi	12
15.1 La regola “la documentazione segue il codice”	12
15.2 Comandi	12

1 Scopo di questo documento

Questo *non* è il report d'esame (che vive in `report/main.tex` e va scritto in inglese a progetto finito). È una guida di studio in italiano del software costruito finora: spiega **che cosa è stato fatto**, **dove si trova ogni componente**, **come funziona il flusso di controllo**, **come si aggiungono nuove strategie** e **quali sono gli step futuri e le scelte aperte**. L'obiettivo è metterti in condizione di leggere il codice con una mappa in mano e di decidere quali migliorie sperimentare.

Convenzione: tutti i percorsi sono relativi alla radice del progetto `asa-deliveroo-agent/`. I termini del corso (belief, desire, intention, plan) sono lasciati in inglese perché corrispondono ai nomi nel codice.

2 Il progetto in sintesi

2.1 Requisiti d'esame coperti

Il progetto realizza l'infrastruttura per i quattro pilastri dell'esame (pesi di valutazione tra parentesi):

- **Agente A — BDI (30%)**: agente con revisione di belief e intention, strategia di gioco selezionabile, validato su scenari Challenge 1. Entry point: `src/main-bdi.js`.
- **Agente B — LLM + coordinamento (30%)**: agente che interpreta le richieste degli agenti-missione, adatta la strategia e coordina con l'Agente A. Entry point: `src/main-llm.js`.
- **PDDL (20%)**: planner PDDL integrato come membro della plan library per l'intention `go_to`, opzionale a runtime. File: `src/planning/PddlPlanner.js`.
- **Report + presentazione (20%)**: scheletro LaTeX in `report/` con commenti-guida per ogni sezione.

2.2 Stato attuale (validato)

Il software è **funzionante e testato**:

- **Smoke test offline (npm test)**: assertion su grafo, pathfinding, revisione belief, strategie, parsing missioni, generazione problemi PDDL e normalizzazione ack. Non richiede server né rete.
- **Test live** (11 giugno 2026, server locale, mappa `empty_10`, 60 secondi): la strategia `reward-distance` ha totalizzato **800 punti con 30 parcel consegnati**. Il primo test (punteggio 173) ha rivelato un bug reale poi corretto (sezione [12](#)).

3 Architettura generale

3.1 Il ciclo di controllo BDI

L'architettura segue il modello dei laboratori del corso: *osserva* → *rivedi i belief* → *genera opzioni* → *seleziona/rivedi l'intention* → *pianifica* → *agisci*.

```
eventi socket (map, config, you, sensing, msg)
      |
      +-----v-----+
      | BeliefBase | grafo mappa, io, parcel, agenti,
      +-----+-----+ config, missione, compagno, claim
```

OptionGenerator	 (che cosa È POSSIBILE fare)
Strategy	 (che cosa CONVIENE fare - intercambiabile)
IntentionRevision	 (commit/replace l'LLM scrive qui: con isteresi) vincoli e goal di missione nei belief
PlanLibrary	(COME farlo: GoPickUp, DeliverCarried, GoToMissionTarget, Explore, Wait, PddlGoTo -> FollowPathGoTo)
ActionExecutor	 (move/pickup/putdown serializzati, gate "semaforo rosso")

3.2 Principi di design (da rispettare nelle evoluzioni)

1. **Infrastruttura vs strategia:** le strategie *decidono* (ritornano l'opzione migliore), l'infrastruttura *esegue* (pianifica, muove, valida, logga). Mai mescolare i due livelli.
2. **Movimento deterministico:** pathfinding BFS sul grafo, nessuna decisione LLM nel ciclo passo-passo.
3. **LLM ad alto livello:** interpreta le missioni e produce *strutture dati validate* che entrano nei belief; non chiama mai direttamente le API di gioco.
4. **Azioni serializzate:** il server rifiuta (con penalità) un'azione inviata mentre la precedente è in corso; ogni azione passa da un'unica coda di promise.
5. **Niente costanti magiche:** tutti i parametri stanno in `src/config.js` o nelle opzioni dei costruttori delle strategie.
6. **Mai fidarsi di una sola forma di ack:** i server del corso differiscono dai tipi dichiarati nell'SDK (verificato dal vivo).

4 Mappa del repository

Percorso	Responsabilità
<code>src/main-bdi.js</code>	Entry point Agente A; costruisce e avvia l'intero runtime (funzione <code>buildAgentRuntime</code> riusata anche dall'Agente B).
<code>src/main-llm.js</code>	Entry point Agente B; runtime BDI + livello LLM (interprete missioni, risposta alle domande, inoltre al compagno).
<code>src/config.js</code>	Configurazione runtime centralizzata (env + default documentati).
<code>src/core/BeliefBase.js</code>	Stato del mondo e revisione dei belief; stato missione; riconciliazione.
<code>src/core/OptionGenerator.js</code>	Genera le opzioni candidate e i criteri di validità.
<code>src/core/Intention.js</code>	Un'opzione a cui l'agente si è impegnato; prova i piani in ordine.

<code>src/core/IntentionRevision.js</code>	Politica replace con isteresi; loop che consuma le intention.
<code>src/core/PlanLibrary.js</code>	Tutti i piani eseguibili + assemblaggio della libreria di default.
<code>src/core/ActionExecutor.js</code>	Serializzazione azioni; gate di movimento (red light).
<code>src/core/AgentLoop.js</code>	Collante: eventi → belief → deliberazione.
<code>src/strategies/StrategyBase.js</code>	Interfaccia strategia (<code>utility</code> , <code>selectOption</code>).
<code>src/strategies/*Strategy.js</code>	Le 4 strategie esistenti (sez. 8).
<code>src/strategies/index.js</code>	Registro: id → classe; <code>createStrategy()</code> .
<code>src/planning/GridGraph.js</code>	Grafo orientato della mappa; frecce; blocchi statici e temporanei.
<code>src/planning/PathPlanner.js</code>	BFS; distanze da me; distanza-consegna precalcolata; helper di scoring.
<code>src/planning/PddlPlanner.js</code>	Generazione problema PDDL dai belief + solver online; logging.
<code>src/planning/pddlDomain.js</code>	Dominio STRIPS leggibile + mappa azione→direzione.
<code>src/communication/MessageTypes.js</code>	Busta del protocollo asa-team-v1 e tipi di messaggio.
<code>src/communication/TeamProtocol.js</code>	Handshake, heartbeat, claim, mission-update, ack.
<code>src/llm/LlmClient.js</code>	Wrapper endpoint OpenAI-compatibile (LiteLLM del corso); lazy.
<code>src/llm/MissionInterpreter.js</code>	Messaggio missione → oggetto strutturato (LLM o fallback regex).
<code>src/llm/prompts.js</code>	Prompt di sistema (interpretazione missioni; tool-agent futuro).
<code>src/llm/tools.js</code>	Registro tool validati ad alto livello (pattern lab8).
<code>src/metrics/MetricsCollector.js</code>	Contatori e timeline punteggio di una run.
<code>src/metrics/RunLogger.js</code>	Log JSON-lines per run + scrittura riassunto risultati.
<code>src/utils/</code>	manhattan , sleep , chiavi tile, conversione clock-event, normalizeIdList , estrazione JSON.
<code>scripts/run-bdi.js</code> , <code>run-llm.js</code>	Lancio agenti con flag CLI che sovrascrivono l'env.
<code>scripts/run-experiment.js</code>	Run a tempo: avvia, gioca, scrive risultati, esce.
<code>scripts/smoke-test.js</code>	Test offline completo (npm test).
<code>experiments/</code>	logs/ (JSON-lines) e results/ (riassunti); entrambi git-ignored.
<code>report/</code>	Report d'esame (scheletro) + questo documento.
<code>notebooks/</code>	Solo analisi offline dei risultati (mai runtime).
<code>CLAUDE.md</code>	Regole del progetto, inclusa la regola "la documentazione segue il codice".

5 Il nucleo BDI in dettaglio

5.1 BeliefBase — lo stato del mondo

`src/core/BeliefBase.js` è l'unica fonte di verità. Campi principali: `me` (posizione/punteggio, autoritativi solo via evento `you` e `ack` dei move), `graph` (sez. 6), `parcels`, `agents`, `tileLastSeen`, `config` (config di gioco del server: mai hardcodare!), `mission`, `teammate`, `claims`.

Principi di revisione implementati (dal lab2 e dalla knowledge base):

- **Timestamp su tutto:** ogni belief dinamico ha `lastSeen`.
- **Proiezione del decay:** la reward ricordata è un limite superiore; `projectedReward()` sottrae un punto per ogni *tick intero* di decay trascorso da `lastSeen` (semantica a scatti del server; fix di precisione dell'11/06).
- **Evidenza negativa:** un parcel ricordato viene cancellato solo quando il suo tile *rientra nel campo visivo* e il parcel non c'è più. L'assenza di sensing non è evidenza di assenza.
- **Riconciliazione** (“la mia conoscenza era sbagliata, non il mondo”): `clearCarried()` dimentica i parcel che credevo di trasportare quando il server mi smentisce; `markTilePickedUp()` marca come trasportati i parcel del mio tile quando l'ack di pickup non contiene id utilizzabili.

Lo stato missione (`mission`) è il canale con cui l'LLM influenza il BDI: `setMission()` traduce una missione strutturata in effetti concreti — tile vietati bloccati nel grafo, tile di consegna esclusi, politiche di putdown (`deliverExactly`, `deliverMaxValue`), gate di movimento (`movementAllowed`), goal posizionali (`mission.active`).

5.2 OptionGenerator — che cosa è possibile

Genera le opzioni candidate ad ogni deliberazione: `go_pick_up` (un parcel libero, non reclamato dal compagno, con reward proiettata > 0), `deliver_carried` (se trasporto qualcosa), `go_to_mission_target` (se c'è una missione attiva con coordinate), più i fallback `explore` e `wait`. Espone anche `isStillValid(option, beliefs)`: le condizioni di abbandono (*achieved / impossible / not worthwhile*) usate dal loop di revisione.

Dove intervenire: se una nuova strategia ha bisogno di un nuovo tipo di obiettivo (es. “camping su uno spawner”), il tipo di opzione si aggiunge qui, il piano corrispondente in `PlanLibrary.js`, e la strategia gli assegna un'utilità.

5.3 IntentionRevision — impegno con isteresi

Politica *replace* raffinata: l'agente è impegnato su **una** intention alla volta; una nuova opzione la sostituisce solo se

$$utility_{nuova} > utility_{corrente} + margine$$

con margine additivo configurabile (`agent.hysteresisMargin`, default $5 \approx 5$ punti di reward). Questo evita il *thrashing* fra due parcel di valore quasi identico. Ripresentare la stessa opzione aggiorna solo l'utilità (per confronti freschi) senza riavviare l'intention. `abortCurrent()` interrompe un'intention diventata invalida (il check lo fa `AgentLoop.deliberate()` ad ogni ciclo).

5.4 Intention e PlanLibrary — dal volere al fare

Una **Intention** prova *in ordine* tutte le classi di piano applicabili; se un piano fallisce (lancia un'eccezione) si passa al successivo; se falliscono tutti, l'intention fallisce e la deliberazione successiva riparte da capo. È il meccanismo con cui “i fallimenti vengono riportati al livello delle intention”. Piani esistenti:

- **GoPickUp**: sub-intention `go_to + pickup`; `ack` vuoto $\Rightarrow$ gara persa, belief cancellato, fallimento.
- **DeliverCarried**: consegna al delivery più vicino; selezione *mission-aware* dei parcel da posare (esattamente $N / \text{valore} \leq \text{soglia}$); riconciliazione su `ack` vuoto.
- **GoToMissionTarget**: raggiunge il target di missione più vicino; `deliver_at` posa i parcel; `holdAtTarget` attende (placeholder 5s, da sostituire con sincronizzazione esplicita).
- **Explore**: va allo spawner visto meno di recente (massimizza informazione per tile percorso).
- **Wait**: pausa breve (ultima spiaggia).
- **PddlGoTo $\rightarrow$ FollowPathGoTo**: registrati in quest'ordine; se il PDDL è disabilitato o fallisce, il BFS subentra automaticamente.

FollowPathGoTo è il cuore deterministico: ricalcola il path BFS, esegue un passo alla volta, e su un move fallito fa `moveRetries` tentativi (l'ostacolo dinamico può spostarsi); se il blocco persiste, marca il tile come *soft-blocked* per `softBlockMs` millisecondi (il prossimo BFS ci gira attorno) e fallisce il piano.

5.5 ActionExecutor — serializzazione e semaforo

Tutte le azioni di gioco passano da un'unica catena di promise: la successiva parte solo dopo l'ack della precedente (il server rifiuta con penalità le azioni concorrenti — *ActionMutex*). Il `movementGate` è un predicato sostituibile: quando la missione “red light” chiude il gate, `move()` *attende* invece di fallire — l'intention si congela e riprende al verde, che è esattamente il comportamento premiato.

5.6 AgentLoop — il collante

Collega gli eventi socket ai belief, attende mappa e identità, avvia il loop di revisione e delibera: ad ogni evento `you/sensing`, dopo ogni intention conclusa, e comunque ogni `deliberationIntervalMs` (250 ms) — necessario perché il sensing arriva *solo quando qualcosa cambia*: il silenzio non deve congelare l'agente.

6 Navigazione deterministica

6.1 GridGraph — la mappa come grafo orientato

Costruito una volta dall'evento `map`. Punti chiave:

- **Digrafo dal primo giorno**: i tile freccia ($\uparrow \rightarrow \downarrow \leftarrow$) vietano l'ingresso *contro* la freccia; la regola è codificata omettendo l'arco. Le mappe Challenge 1 n. 4–6 (circuiti a senso unico) funzionano senza modifiche.
- **Blocchi statici**: `blockTiles()` per i vincoli di missione (“non passare per (x,y)”); `setForbiddenDeliveries` esclude tile di consegna vietati.
- **Soft block**: tile temporaneamente evitati (agente fermo sul percorso) con scadenza automatica.
- **Distanza-consegna precalcolata**: BFS multi-sorgente dai delivery sugli *archi invertiti* (corretto sul digrafo: con i sensi unici la distanza *verso* un delivery non è simmetrica). Lookup $O(1)$ usato dalle utilità.

6.2 PathPlanner — BFS e helper di scoring

`shortestPath` (BFS, costi unitari), `distancesFrom` (una BFS sola per prezzare *tutte* le opzioni di una deliberazione), `nearestDelivery`, e `scoringHelpers()` che fornisce alle strategie: `distanceTo(x,y)`, `deliveryDistanceFrom(x,y)`, `decayPerTile` (reward persa per passo per parcel trasportato = $\text{movement_duration} / \text{decay_interval}$).

7 PDDL

Scelta architetturale (motivata nella knowledge base): il PDDL è un **membro della plan library**, non il loop esterno. Il mondo cambia ad ogni frame e il solver online ha latenze di secondi: ripianificare via HTTP ad ogni tick non è praticabile; servire l'intention `go_to` con un piano STRIPS sì, ed è un'integrazione “significativa” richiesta dall'esame.

- `pddlDomain.js`: dominio leggibile con predicati (`at ?t`) e archi direzionali (`up/down/left/right ?from ?to`); 4 azioni `move-*`. I sensi unici sono codificati gratis: l'ingresso vietato semplicemente non ha il predicato-arco.
- `PddlPlanner.js`: genera il problema dai belief (regione raggiungibile via BFS, con limite `pddl.maxTiles`), chiama `onlineSolver` di `@unitn-asa/pddl-client` (caricato lazy: senza pacchetto o con `PDDL_ENABLED=false` non se ne paga il costo), converte i passi in direzioni, **logga ogni chiamata e ogni fallimento** (`pddl_plan` / `pddl_failure`) — dati pronti per il confronto BFS-vs-PDDL nel report.

Evoluzione prevista: estendere il dominio con azioni `pickup/putdown` (pianificare intere sequenze raccogli-e-consegna) o con la spinta delle casse (`domain-deliveroojs-crates.pddl` come base) per le mappe di pratica sokoban-like.

8 Le strategie

8.1 L'interfaccia

`StrategyBase.js`: una strategia implementa `utility(option, beliefs, helpers)` (e/o sovrascrive `selectOption` per logiche non a utilità). La selezione di default è l'argmax delle utilità; le utilità vengono scritte *sulle opzioni* perché la revisione le usa per l'isteresi. Convenzione di scala: “punti di reward attesi”, così opzioni di tipo diverso restano confrontabili. La `StrategyBase` dà già a *tutte* le strategie il rispetto delle missioni posizionali (i bonus $\pm 200 \dots \pm 1000$ dominano il reddito da parcel).

8.2 Le quattro strategie esistenti

Id	Idea e formula
greedy-nearest	Baseline di controllo: insegue il parcel raggiungibile più vicino ($u = 1000 - d$); consegna solo quando non c'è altro da raccogliere ($u = 500 - d_{\text{delivery}}$). Ignora valore e decay di proposito.
reward-distance	Massimizza il valore <i>consegnato</i> : $u_{\text{pick}} = \text{proj}(p) - (d_{me \rightarrow p} + d_{p \rightarrow del}) \cdot \text{decay} \cdot (c + 1)$; $u_{\text{del}} = \sum \text{proj}(\text{carried}) - d_{me \rightarrow del} \cdot \text{decay} \cdot c - \text{deliverBias}$, dove c = parcel trasportati e <code>deliverBias</code> (default 10) incoraggia il batching.

delivery-threshold	Come sopra, ma accumula fino a soglia (<code>minCarried</code> , default 3, o <code>minValue</code>) e poi la consegna domina (+1000). Pensata per mappe dove le consegne sono il collo di bottiglia (es. 26c1_5: 91 spawner, 4 delivery).
mission-aware	<code>reward-distance</code> + obbedienza allo stato missione: peso configurabile sui goal di missione (<code>missionWeight</code>); boost (+200) alla consegna quando la politica <code>deliverExactly/deliverMaxValue</code> è soddisfacibile. Default dell'Agente B.

8.3 Come aggiungere una nuova strategia (passo-passo)

1. Crea `src/strategies/MiaStrategia.js`:

```
import { StrategyBase } from './StrategyBase.js';
// oppure: extends RewardDistanceStrategy per ereditarne le utilita'

export class MiaStrategia extends StrategyBase {
  static id = 'mia-strategia';

  constructor(options = {}) {
    super(options);
    this.mioParametro = options.mioParametro ?? 42; // niente costanti magiche
  }

  utility(option, beliefs, helpers) {
    if (option.type === 'go_pick_up') {
      const parcel = beliefs.parcels.get(option.parcelId);
      const d = helpers.distanceTo(option.x, option.y); // distanza VERA (BFS)
      if (!Number.isFinite(d)) return -Infinity; // irraggiungibile
      return beliefs.projectedReward(parcel) - d * helpers.decayPerTile;
    }
    return super.utility(option, beliefs, helpers); // missioni/explore/wait
  }
}
```

2. Registrala in `src/strategies/index.js`: un import + una riga nella lista `STRATEGY_CLASSES`.
3. Lanciala: `node scripts/run-bdi.js -strategy mia-strategia` (o `STRATEGY=mia-strategia` nel `.env`).
4. Confrontala: `node scripts/run-experiment.js -strategy mia-strategia -duration 180 -label 26c1_3`, almeno 5 run per coppia (mappa, strategia), confronto sulle medie.

Strumenti a disposizione dentro `utility`: tutti i belief (`beliefs.parcels`, `beliefs.agents`, `beliefs.carried()`, `beliefs.projectedReward(p)`, `beliefs.mission`, `beliefs.teammate`, `beliefs.claims`) e gli helper geometrici (`distanceTo`, `deliveryDistanceFrom`, `decayPerTile`). Per logiche non a punteggio (es. macchine a stati) sovrascrivi `selectOption` e ritorna direttamente l'opzione scelta (ricordati di assegnarle `option.utility`, serve all'isteresi).

9 L'Agente B: livello LLM

9.1 Flusso

`main-llm.js` costruisce *lo stesso* runtime BDI dell'Agente A (deve pur sempre muoversi e consegnare) e ci aggiunge un listener sui messaggi non-protocollo: ogni messaggio (tipicamente lo `shout`

di un agente-missione) passa da `MissionInterpreter.interpret()`; il risultato strutturato viene applicato ai propri belief (`setMission`) e inoltrato all'Agente A via `protocol.sendMissionUpdate()`. Le missioni `question_answer` (es. "Calcola $5 * (5 + 3) / 2$ ") vengono risposte direttamente in chat senza muoversi (livello 1 della Challenge 2).

9.2 MissionInterpreter — doppio binario

- **Binario LLM** (se configurato in `.env`): un prompt (`prompts.js`) estrae `kind`, `coordinate`, `segno del bonus`, `soglie`; l'output JSON è **validato** (`#validate`) prima di toccare i belief; output invalido $\Rightarrow$ fallback.
- **Binario deterministico** (sempre disponibile): regex sul catalogo missioni della Challenge 2 — `go_to`, `deliver_at` (anche in forma vietata), `question_answer` (con valutatore aritmetico sanificato), `deliver_exactly_n`, `deliver_less_value_than`, `one_pickup_another_deliver`, `red_light_green_light`.
- **Eccezione di latenza**: i cambi di stato RED LIGHT/GREEN LIGHT sono *sempre* parsati senza LLM (un round-trip di rete durante il rosso costerebbe penalità).

9.3 Tools (per esperimenti futuri)

`tools.js` contiene il registro in stile lab8 (`get_state`, `set_mission`, `block_tiles`, `set_movement_allowed`, `send_to_teammate`, `reply_to_agent`) con dispatch validato (`executeTool`). Non è nel flusso di default: serve a sperimentare un loop in cui l'LLM orchestra i tool per richieste aperte. Volutamente **non esiste** un tool "muoviti di un passo".

10 Coordinamento A-B

Protocollo `asa-team-v1` (`MessageTypes.js`): busta JSON `{protocol, type, payload, from, ts}`. I messaggi non-busta (es. shout delle missioni) vengono ignorati dal protocollo e gestiti dal livello LLM.

Tipo	Payload	Uso
hello	<code>{name, isReply?}</code>	Scoperta del compagno: shout all'avvio; chi riconosce il nome atteso (<code>TEAMMATE_NAME</code>) registra l'id e risponde via <code>say</code> .
position	<code>{x, y, carrying, intention}</code>	Heartbeat ~ 1 s (raddoppia l'osservazione effettiva del team).
intention	<code>{key, type}</code>	Annuncio intention corrente.
claim	<code>{parcelId}</code>	Deconflitto target: primo che reclama vince; l' <code>OptionGenerator</code> dell'altro scarta il parcel.
mission-update	<code>{mission}</code>	B $\rightarrow$ A: missione strutturata da applicare ai belief.
request-help	<code>{reason, x?, y?}</code>	Richiesta d'aiuto (gestione strategica: TODO).
handover	<code>{parcelId, x, y, role}</code>	Passaggio di parcel (<code>26c2_8</code>) — coreografia completa: TODO.
ack	<code>{about}</code>	Conferma; via callback di <code>ask</code> se presente.

Nota operativa: `say` per i flussi normali; `ask` (bloccante, timeout server 1 s) riservato ai punti di sincronizzazione, con timeout proprio lato client.

11 Metriche, log ed esperimenti

- **RunLogger**: un file JSON-lines per agente per run in `experiments/logs/` (`{t, event, ...}`). Eventi: punteggio, pickup/consegne (con `count` e `ids` normalizzati), intention avviate/concluse/fallite/abortite, piani falliti, chiamate/fallimenti PDDL, interpretazioni LLM, messaggi.
- **MetricsCollector**: contatori in memoria + timeline del punteggio; `summary()` scritto in `experiments/results/` a fine run.
- **run-experiment.js**: run a tempo riproducibile: `-agent bdi|llm -strategy <id> -duration <s> -label <scenario>`.
- **Metodologia**: gli spawn sono casuali $\Rightarrow$ almeno 5 run per coppia (scenario, strategia) e confronto su $\text{media} \pm \text{dev. standard}$ (stesso pattern del benchmark del professore).
- Gli output grezzi sono **git-ignored**: i numeri che servono al report vanno copiati nei sorgenti del report o nei README.

12 Storia della validazione (e lezioni)

12.1 Il bug dei “parcel fantasma”

Primo test live (60 s, `empty_10`): punteggio 173, ma 352 cambi di intention e 336 putdown falliti. Diagnosi dai log: gli **ack del server non contenevano il campo id** (forma diversa dai tipi dichiarati nell'SDK). Risultato: dopo ogni consegna il belief “sto trasportando X” restava vivo (parcel fantasma) e l'agente riprovava il putdown all'infinito finché la proiezione del decay non azzerava il fantasma.

Fix (tre pezzi, tutti in chiave “belief reconciliation”): `normalizeIdList()` accetta ack come oggetti `{id}`, stringhe o forme miste; `markTilePickedUp()` come fallback del pickup; `clearCarried()` quando il server smentisce il carry. Secondo test: **punteggio 800, 30 consegne, zero retry futili**. La coppia di run è un esempio già pronto, con dati, del valore della revisione dei belief per il report (sezione belief revision).

12.2 Fix del decay (contributo Codex)

`projectedReward` sottraeva frazioni di tick ($\lfloor r - t/T \rfloor$); il server decrementa a scatti interi. Ora: $r - \lfloor t/T \rfloor$, con guardia `Math.max(0, elapsed)` contro skew d'orologio. Pinnato da assertion nello smoke test.

12.3 Lezione generale

Mai fidarsi dei tipi dichiarati: i server del corso (locale vs challenge) possono differire dall'SDK. Ogni dato che entra nei belief passa da normalizzazione e, dove possibile, da riconciliazione con la realtà osservata.

13 Step futuri

In ordine di priorità suggerito:

1. **Tuning e validazione Challenge 1** (pilastro da 30%): girare le 8 mappe `26c1_*`; tarare `deliverBias`, `minCarried`, `hysteresisMargin` per mappa; verificare i circuiti a senso unico (4-5) e la competizione con l'NPC onnisciente.

2. **Utilità opponent-aware**: se un avversario visibile è più vicino del nostro agente al parcel target, abbandonarlo (l’NPC “intelligent” è onnisciente: la gara si vince solo in velocità).
3. **Coreografia handover (26c2_8)**: negoziare il tile di rendezvous via `ask`, sequenziare putdown-prima-del-pickup (due agenti non condividono il tile), scambiarsi gli id dei parcel.
4. **Go-to-and-wait di squadra (26c2_10)**: sostituire l’attesa fissa di 5s con sincronizzazione esplicita (`position + ack`).
5. **PDDL esteso**: azioni pickup/putdown nel dominio; esperimento BFS-vs-PDDL (latenza, lunghezza piano, fallimenti) per la sezione PDDL del report.
6. **Tool loop LLM** (opzionale): l’LLM orchestra i tool di `tools.js` per richieste aperte (pattern lab8 07).
7. **Esperimenti sistematici + report**: ≥ 5 run per (mappa, strategia); notebook di analisi; riempire le sezioni del report seguendo i commenti-guida.

14 Scelte aperte e parametri da esplorare

14.1 Manopole esistenti (nessuna modifica al codice richiesta)

Parametro	Default	Che cosa governa / che cosa provare
<code>hysteresisMargin</code>	5	Stabilità dell’impegno: alzarlo riduce i cambi di target, abbassarlo rende l’agente più opportunist. Confrontare i <code>intentionChanges</code> nei risultati.
<code>deliverBias</code>	10	Propensione al batching di <code>reward-distance</code> : su mappe con decay veloce conviene abbassarlo.
<code>minCarried / minValue</code>	3 / ∞	Soglie di <code>delivery-threshold</code> : idealmente derivarle dal config dello scenario (<code>parcels.max</code> , velocità di decay) invece che fissarle.
<code>missionWeight</code>	1	Aggressività sulle missioni di <code>mission-aware</code> .
<code>deliberationIntervalMs</code>	250	Frequenza di riconsiderazione: scenari dinamici $\rightarrow$ più alta; stabili $\rightarrow$ più bassa (meno overhead).
<code>moveRetries / softBlockMs</code>	2 / 3000	Pazienza contro ostacoli dinamici vs velocità di re-routing.
<code>pddl.enabled / maxTiles</code>	false / 1600	Attivazione PDDL e limite dimensione problema.

14.2 Idee strategiche da prototipare

- **Partizionamento della mappa** per il team (regioni di Voronoi sui delivery) per non farsi concorrenza — da misurare contro il semplice claim.
- **Camping sugli spawner** in mappe a spawn lento: piazzarsi al centroide degli spawner invece di esplorare.
- **Stima del movimento avversario** dalle posizioni successive nel sensing (i belief agents hanno già `lastSeen`).
- **Soglie auto-adattive**: leggere `parcels.max`, `generation_event`, `decaying_event` dal config e derivare i parametri di batching per scenario.
- **Sfruttare il throttling degli spawn**: i parcel trasportati contano nel limite `parcels.max` $\Rightarrow$ accumulare affama la mappa (rilevante in 26c1_4/5 con `max=5`).

14.3 Domande aperte da verificare a runtime

Dalla knowledge base (`context/game_knowledge/06`): la capacità è davvero non applicata ai player sul server delle challenge? Il sensing include i parcel trasportati da me? (localmente: sì, ma via ack ci arriviamo comunque). Latenza fra evento-missione e accredito del bonus? Versione del server delle challenge vs locale? Convieni un probe-agent che logga tutti gli eventi grezzi per 5 minuti per mappa.

15 Regole di lavoro e comandi rapidi

15.1 La regola “la documentazione segue il codice”

Codificata in `CLAUDE.md` e **obbligatoria**: ogni modifica al comportamento del codice aggiorna nello stesso cambiamento (1) il `README.md` (implementato/assunzioni), (2) `scripts/smoke-test.js` (assertion sul nuovo comportamento; `npm test` deve passare offline), (3) `experiments/README.md` se cambiano eventi/contatori/formati, (4) `notebooks/README.md` se cambia il formato dei risultati, (5) i commenti-guida in `report/sections/*.tex`, (6) i commenti JSDoc. Una modifica non è completa finché la checklist non è stata percorsa.

15.2 Comandi

<code>npm test</code>	<code># smoke test offline</code>
<code>node scripts/run-bdi.js --strategy <id></code>	<code># Agente A</code>
<code>node scripts/run-llm.js --name agentB</code>	<code># Agente B</code>
<code>node scripts/run-experiment.js --strategy <id> \</code>	
<code> --duration 180 --label <scenario></code>	<code># run a tempo con risultati</code>

Variabili d'ambiente principali (vedi `.env.example` commentato): `HOST`, `TOKEN` (salvare quello stampato al primo avvio per mantenere l'identità), `NAME`, `TEAMMATE_NAME`, `STRATEGY`, `PDDL_ENABLED`, `LITELLM_*`.

Fine del documento. Aggiornarlo quando l'architettura evolve: fa parte della documentazione coperta dalla regola di cui sopra.